

Communications of the ACM**Research and Advances**[Computing Applications](#)

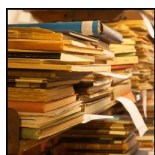
Contributed Articles

(Re)Use of Research Results (Is Rampant)

Prior pessimism about reuse in software engineering research may have been a result of using the wrong methods to measure the wrong things.

By Maria Teresa Baldassarre, Neil Ernst, Ben Hermann, Tim Menzies, and Rahul Yedida

Posted Feb 1 2023



According to Popper,²³ the ideas we can most trust are those that have been the most tried and tested. For that reason, many of us are involved in this process called “science,” which produces trusted knowledge by sharing one’s ideas and trying out and testing the ideas of others. Science and scientists form communities where people do each other the courtesy of curating, clarifying, critiquing, and improving a large pool of ideas.

Feedback

[Back to Top](#)**Key Insights**

- Researchers must take back control over how our products are shared.
- Results should not be locked away behind paywalls that block access to results and enshrine outdated views on science. Using open source tools, research communities can map the real patterns of inference between their work.
- For example, in SE, researchers often share many artifacts, ranging from ideas to paper-based methods, datasets, and tools. While exact replication of results is rare, we can report just from one year that there are hundreds of cases where one researcher used some but not all of the artifacts from other work.
- We ask others to join us in this effort to accurately record the reality of 21st century science.



According to this definition, one measure of a scientific community’s health is how much it reuses results. By that measure, the software engineering research community might seem to be very unhealthy. Da Silva et al. reported that from 1994 to 2010, only 72 studies had been replicated by 96 new studies.¹⁰ In February 2022, as a double-check for da Silva’s conclusion, we queried the ACM Portal for products from the International Conference on Software Engineering (ICSE), that field’s premier conference. Between 2011 and 2021, only 111 out of the 8,774 ICSE research entries were labeled as ‘available,’ 74 as ‘reusable,’ 24 as ‘functional,’ and none as ‘replicated’ or ‘reproduced’ reuse (see [Table 1](#) for a definition of those terms). Put another way, according to the ACM Portal, only 2.4% of the ICSE publications are explicitly associated with any kind of reuse. Worse still, according to that report, there were no replicated or reproduced results from ICSE in the last decade.

Available	Functional	Reusable	Reproduced	Replicated
				
In a public repository with a long-term retention policy. A DOI needs to be provided.	Artifacts are documented, consistent, complete, exercisable, and include evidence of verification and validation.	Functional, significantly exceed minimal functionality.	Results of this paper have been reproduced by a different team using the original artifact.	Results of this paper have been replicated by a different team without the original artifact.

Table 1. Badges such as the ones shown in this table are currently awarded at conferences.² This table is based on ACM’s badge program, however, analogous badges are used at other conferences. Images used by permission of the Association for Computing Machinery.

We argue that the reuse “problem” is more apparent than real—at least in software engineering. We describe a successful approach to recording research reuse where teams of researchers from around the world read 170 recent (2020) conference papers from software engineering. This work generated the “reuse graph” in [Figure 1](#), in which each edge connects papers to the prior work they are (re)using. As we will discuss, when compared to other community monitoring methods (for example, artifact tracks or bibliometric searches^{5,12,19}), these reuse graphs require less effort to build and verify. For example, it took around 12 minutes per paper for our team from Hong Kong, Canada, the U.S., Italy, Sweden, Finland, and Australia to apply this reuse graph methodology to software engineering.⁴



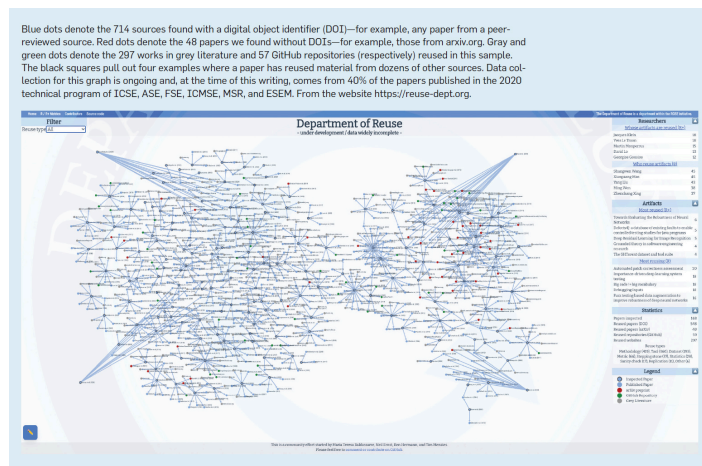


Figure 1. The 1,635 arrows in this diagram connect reusers to the reused.

The rest of this article discusses generating, applying, and the value of our reuse graphs. Before beginning, we offer the following introductory remark. This article is written as a protest, of sorts, against how we currently assess science and scientific output. This article’s authors have worked as researchers for decades, supervising graduate students and organizing prominent conferences and journals. Based on that experience, we assert that researchers do more than write papers. Rather, we are all engaged in long-term stewardship of ideas; as part of that stewardship, we generate more than just papers. Yet, of all our products, it is only our papers that are used, mostly in some annual bibliometric analysis of our worth. We view this as an inadequate way to measure what researchers do.

The problem, we think, is in the very term “bibliometric.” This term is heavily skewed toward publications and monographs and the kinds of things we can easily store in the repositories of our professional societies—for example, IEEE Xplore and ACM Portal. In fact, the term “bibliométrie” was first used by Paul Otlet in 1934²⁵ and was defined as “the measurement of all aspects related to the publication and reading of books and documents.”

This article is written as a protest, of sorts, against how we currently assess science and scientific output.

Subsequent definitions tried to broaden that definition. For example, the anglicized version “bibliometrics” was first used by Alan Pritchard in his 1969 paper, “Statistical Bibliography or Bibliometrics?”, where he defined the term as “the application of mathematics and statistical methods to books and other media of communication.”²⁴ But what we are observing in 2022 is that “other media of communication” in software engineering (and other fields) is far broader than just the products stored in the repositories of our professional societies. For example, researchers might use the results of papers, follow guidance from one paper in their own work, or download data or code used on another paper (and then use locally). We argue that all such

downloads or guidance-following are examples of reuse, since all are examples of members in our research community reusing products from other research in their own work (for a more exact categorization of the types of reuse we are studying, please see our section Studying Reuse).

It is all too easy to propose a broader definition for how scholars reuse and communicate their products. Such a new definition is practically useless unless we can propose some method to collect data on that new definition. We suggest that our new definitions can be operationalized via crowdsourced methods.

[Back to Top](#)

Capturing Reuse

There are many methods to map the structure of SE research, such as (a) manual or automatic citation searchers or (b) “artifact evaluation committees” that foster the generation and sharing of research products. Such studies can lag significantly behind current work. For example, in our own prior citation analysis of SE,¹⁹ we only studied up to 2016. The study itself was conducted in 2017, but not fully published till 2018. Given the enormous effort required for that work, we have vowed never to do it again.

Reuse graphs, on the other hand, are faster to update since the work of any individual working on these graphs is minimal. Other reasons for favoring reuse graphs are that they are community comprehensible, verifiable, and correctable. All the data used for our reuse graphs is community-collected and can be audited at <https://reuse-dept.org>. If errors are detected, issue reports can be raised in our GitHub repository and then corrected. The same may not hold true for studies based on citation servers run by professional bodies and for-profit organizations (see [Table 2](#)). New data can be contributed by anyone either directly supplying data in our format or through a user interface directly on our website for easier access. The resulting issue report is then reviewed and, when necessary, corrected. After a third person successfully inspects the data, it is added to the reuse graph.

EXAMPLE #1
One of the authors has an entry in Google Scholar metrics software systems saying that their “How to” paper has received 80 citations over the past five years at IEEE Transactions on Software Engineering (TSE). That link connects to some other paper, not written by any of us. There is no “help” button at Google Scholar where this error can be reported. This is disappointing, since it is suspected that this mysterious “How to” paper references work that might be the most cited from IEEE TSE in the last five years. That would be a significant achievement, if we could document it, but using Google Scholar, we cannot.
EXAMPLE #2
In our recent large-scale text-mining studies of more than 30,000 SE papers, ¹⁹ we found that papers can appear on our citation server but not on other servers. Also, examples were found where some papers had 20 times the citations on one server than on another. Again, we were unable to contact anyone working on those citation servers to fix those errors.
EXAMPLE #3
Even if the owners of these citation sites can be contacted, their errors may remain unfixed. For example, there is no accepted convention for how to typeset a hyphen. Different venues used no spaces or a single space around hyphens, while others typeset the hyphen as two dashes. Zhou et al. found that papers with hyphens in the title get reported as different papers in different venues, which means that those papers receive fewer citations. ²⁹ Zhou et al. also reports that when they contacted the owners of these citation servers, rather than fix the errors, the owners lobbied for the Zhou et al. paper not to be published.

Table 2. Examples of errors in citation servers.

What is the value of a verified, continually updated snapshot of a current research area? Once our reuse graph covers several years (and not just 2020 conference publications), we foresee several applications:

- Academics can check that their contributions to science are being properly recorded.
- When applying for a promotion or new position, research faculty or industrial workers could document the impact of their work beyond papers, including tools, datasets, and innovative methods.
- Graduate students could direct their attention to research areas that are both very new (nodes from recent years) and very productive (nodes with an unusually large number of edges attached).
- Organizers of conferences could select their keynote speakers from that space of new and productive artifacts.
- Growth patterns might guide federal government funding priorities or departmental hiring plans.
- Venture capitalists could use these graphs to detect emergent technologies, perhaps even funding some of those.
- Conference organizers could check if their program committees have enough members from currently hot topics.
- Further, those same organizers could create new conference tracks and journal sections to service active research communities that are under-represented in current publication venues.
- Journal editors could find reviewers with relevant experience.
- Educators can use the graphs to guide their teaching plans.

[Back to Top](#)

Studying Reuse

In our reuse study, we targeted papers from the 2020 technical programs of six major international SE conferences: ICSE, Automated Software Engineering (ASE), Joint European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE), Software Maintenance and Engineering (ICSME), Mining Software Repositories (MSR), and Empirical Software Engineering and Measurement (ESEM). These conferences were selected using advice from Matthew et al.,¹⁹ but our vision is to expand—for example, by looking at all top-ranked SE conferences. GitHub issues were used to divide up the hundreds of papers from those conferences into “work packets” of 10 papers each. Reading teams were set up from software engineering research teams from around the globe in Hong Kong; Istanbul, Turkey; Victoria, Canada; Gothenburg, Sweden); Oulu, Finland; Melbourne, Australia; and Raleigh, NC, USA. Team members assigned themselves work packets and read the papers in search of examples of reuse listed in the next paragraph. Once completed, a second person (from any of our teams)

performed the same task and checked for consistency. Fleiss Kappa statistics were then computed to track the level of reader disagreement. GitHub issues^b were used to manage this in the open, but raters were asked not to examine previous results. A member of this article's author team then performed a final check on disagreements before including the data into the graph.

Teams were asked to record six kinds of reuse:

1. Most papers must benchmark new ideas against some prior recent state-of-the-art paper. That is, they reuse old papers as steppingstones toward new results.
2. Statistical methods are often reused. Here we do not mean “we use a two-tailed t-test” or some other decades-old, widely used statistical method. Rather, we refer to statistical methods in recent papers that propose statistical guidance for the kinds of analysis seen in SE. Perhaps because this kind of analysis is very rare, this work is highly cited. For example:
 - A 2008 paper, “Benchmarking Classification Models for Software Defect Prediction,”¹⁸ has 1,178 citations
 - A 2011 paper, “A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms,”¹ has 778 citations.
1. Metrics and methodology descriptions that are specific to the research area, including software metrics such as CK metrics or flow metrics as well as research methods such as grounded theory or sampling criteria.
2. Datasets
3. Sanity checks, which justify why a particular approach works or is reasonable to avoid bad data—for example, why to avoid using GitHub stars to select repositories.¹⁶
4. Software packages of the kind currently being reviewed by SE conference AECs (tools and replications).

[Figure 2](#) shows an example. Starting with a paper by Bernal-Cárdenas et al.,⁵ we find among others a reused dataset from Moran et al.,²² tool reuse of FFmpeg and Tensorflow object detection, and several reused methods, including the ConvNet approach described by Simonyan. Readers can follow the URL in the [Figure 2](#) caption for more detailed information.

This figure shows reuse from Bernal-Cárdenas et al.⁷ Edges reflect tool, dataset, and methodology reuse. Red nodes indicate arXiv preprint; green represents a GitHub repository; blue denotes a published paper, and grey indicates other websites or grey literature locations. <https://www.reuse-dept.org/doi/10.1145/3377811.3380328>.

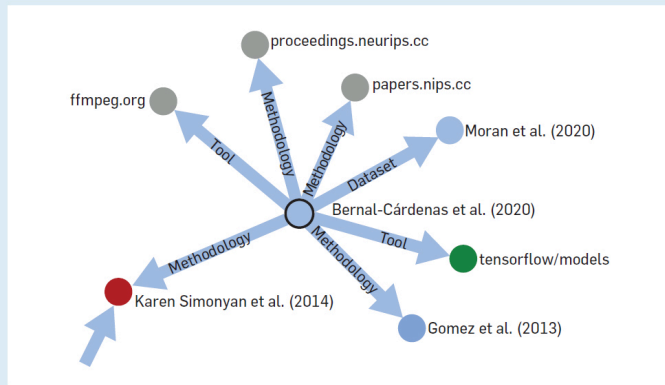


Figure 2. A detailed view of a section of the reuse graph from Figure 1.

We can report that it is not difficult to read papers to detect these kinds of reuse:

- The six types of reuse noted above can be found quickly. Our graduate students report that reading their first paper might take up to an hour. But after two or three papers, the median reading time drops to approximately 12 minutes (see Figure 3a).

In the middle chart, the Fleiss Kappa score⁸ measures the inter-rater reliability among multiple raters when there are categorical ratings to items. We prefer it to Cohen's kappa measure, which only works for two raters. In our case, we had multiple raters assigning a pool of papers the categories "Yes" (signifying that a paper was reused) or "No" (signifying that it was not). This pool was obtained by taking the union of all the papers marked as being reused by a specific paper.

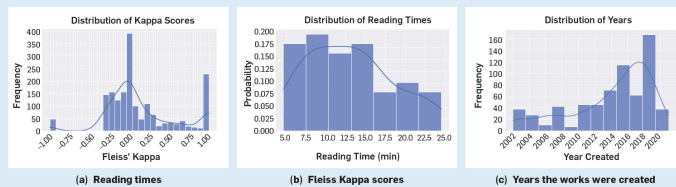


Figure 3. Reading time results, agreement scores, and yearly prevalence of reused papers.

- When we compare the reuse reported by different readers, we get Figure 3b. In our current results, the median Fleiss Kappa score (for reviewer agreement) is 1—that is, very good.
- The one caveat we would add is that graduate students involved in this activity need at least two years of active research experience in their area of study. We base this on the fact that when we tried data collection from a large intro-to-SE graduate subject, the resulting Kappa agreement scores were poor.

The result of this data collection is a directed multi-graph of publications and other forms of dissemination of research artifacts. The edges of this graph are annotated with the type of reuse according to the list above. Reuse metrics for a specific publication (or other form, for example, a GitHub repository) are the in-degree and out-degree measures of the node that represents this publication. When accumulated for the originating authors, individual reuse metrics can be collected. Zooming into the graph on our website, reuse types are visibly annotated at the graph

edges (see [Figure 2](#)). A filter allows a graph to be extracted for a single reuse type out of the multi-graph.

Of course, there are many more items being reused than just the six we have listed.^c It is an open question, worthy of future work, to check if those other items can be collected in this way and, indeed, to refine these categories as understanding changes.

[Back to Top](#)

Related Work

Apart from software engineering,²¹ many other disciplines are actively engaged in artifact creation, sharing, and reuse.^{3,4} Artifacts are useful for building a culture of replication and reproducibility,^{9,17} already acknowledged as important in SE.^{8,19,15,27} Fields such as psychology have had many early results thrown into doubt due to a failure to replicate the original findings.²⁸ Sharing research protocols and data through replication packages and artifacts allows for other research teams to conduct severe tests of the original studies,²⁹ strengthening or rejecting these initial findings.

In medicine, drug companies are mandated to share the research protocols and outcomes of their drug trials, something that has become vitally important recently, albeit not without challenges.¹¹ In physics and astronomy, artifact sharing is so commonplace that large community infrastructures exist solely to ensure data sharing, not least because the governments which fund these costly experiments insist on it.

In more theoretical areas of CS, the pioneering use of preprint servers has enabled ‘reuse’ of proofs, which has been essential to progress. In machine learning, replication is focused on steppingstones, enabled by highly successful benchmarks such as ImageNet.²⁶ However, recent advances with extremely costly training regimens have called replicability into question.^d

In the specific case of SE research, prior to this paper, there was little recorded and verified evidence of reuse. Many researchers have conducted citation studies that find links to highly cited papers—for example, Matthew et al.¹⁹ As stated previously, such studies can lag the latest results. Also, recalling [Table 2](#), we have cause to doubt the conclusions from such citation studies.

From a practical perspective, many conferences have recently introduced AECs to entice reuse and replication. Moreover, authors of accepted conference papers submit software packages that, in theory, let others re-execute that work.^{8,9} These committees award badges, as shown in [Table 1](#).

Artifact evaluation is something of a growth industry in the SE as well as the programming languages (PL) communities, as shown in [Figure 4](#), which presents the increasing number of people evaluating artifacts between 2011 and 2019. One may conclude that such practices make the community more aware of what is available and reusable, and therefore, can become a

potential node of a reuse graph. As such, the source is explicitly made available to any other researcher willing to (re)use it.^{8,14}



Figure 4. Artifact evaluation committee sizes, 2011–2019.¹⁴

Now, the question to be asked is: Are all the people of [Figure 4](#) making the best use of their time? Perhaps not. Most artifacts are assigned the badges requested by the authors, so it might be safe to ask some of the personnel from [Figure 4](#) to, for example, spend less time evaluating conference artifacts and more time working on [Figure 1](#).

But most importantly, it is not clear whether the artifact evaluation process is creating reused artifacts, and therefore, indirectly contributing to the reuse graph concept. Indeed, if we query ACM Portal for “software engineering” and “artifacts” between 2015 and 2020, we find that most of the recorded artifacts are not reused in replications or reproductions.^e Specifically, only 1/20 are reproduced and only 1/50 are replicated.

Perhaps it might be useful to reflect more on what is being reused (as we have done earlier in this article). This is what has motivated our research and led us to create the reuse graph.

[Back to Top](#)

Next Steps for Reuse Graphs

When discussing this work with colleagues, we are often asked if we have assessed it. We reply that, at this stage, this is like asking the inventors of kd-trees⁶ in 1975 how much that method has sped up commercial databases. Right now, we are engaged in community building and have shown that we can create the infrastructure needed to collect our data with very little effort and not much coding. While [Figure 1](#) is a promising start, scaling up requires that we organize a larger reading population. Our goal is to analyze 200 papers in 2022, 2,000 in 2023, and 5,000 in 2024, by which time we would have covered most of the major SE venues in the last five years. After that, our maintenance goal would be to read around 500 papers per year to keep up to date with the conferences (then, we would move on to journals). Based on [Figure 3a](#), and assuming each paper is read by two people, the maintenance goal would be achievable by a team of 20 people working two hours per month on this task. To organize this work, we have created the ROSE Initiative (see the sidebar: The Rose Initiative for more information).

If that work interests you, then there are many ways you can get involved:

- Visit <https://reuse-dept.org> if you are a researcher and wish to check that we have accurately recorded your contribution.
- If you want to apply reuse graphs to your community, please use our tools at <https://github.com/bhermann/DoR/>.
- If you would like to join this initiative and contribute to an up-to-the-minute snapshot of SE research, then please take our how-to-read-for-reuse tutorial,^f and then visit the dashboard at the GitHub site (bhermann/DoR). Find an issue with no one's face on it, and assign yourself a task.

If we take an agile view of SE science, then as researchers we should focus on generating artifacts and rapidly securing critique, curation, and clarification.

We see this effort as one part of the broader open science effort, in addition to helping the community identify the state of the art—for example, patterns of growth in the reuse graph. Among the goals of open science are the desire to increase confidence in published results and an acknowledgment that science produces more types of artifacts than just publications: Researchers also produce method innovations, new datasets, and better tools. If we take an agile view of SE science, then as researchers we should focus on generating these artifacts and rapidly securing critique, curation, and clarification from our peers and the public.



Figure. Watch the authors discuss this work in the exclusive *Communications* video. <https://cacm.acm.org/videos/reuse-of-research>

[Back to Top](#)

[Back to Top](#)

[Back to Top](#)

[Back to Top](#)

Sidebar: The ROSE Initiative

The Rose Initiative (Recognizing and Rewarding Open Science in Software Engineering) is an international, multi-conference workshop that will continually report updates to the software engineering reuse graphs.

- Researchers who reuse the most from other papers will be applauded and awarded an “R-index” (reuse index).
- Researchers who build the artifacts that are most reused will be applauded (even louder) and awarded an “R+-index,” indicating they are producing the artifacts that are most used by the rest of the community.
- Between each conference, the ROSE Initiative will coordinate an international team of volunteers to incrementally update the SE reuse graph. This article’s authors have volunteered to serve as the group’s initial nucleus, and we will hold open elections to grow that steering committee using others from our community.
- This reuse graph will be displayed at a publicly available website (reuse-dept.org), where individual researchers can browse, check their entries, and propose corrections and extensions.
- All reuse reports will be double-checked, and disputed claims will then be triple-checked.
- All the tools used to create the site will be freely available for download. Hence, if the SE community does not like how we are running these reuse graphs, they can use the code and data for a different application.
- Also, researchers from other disciplines can apply our tools to their own community.

References

1. Arcuri, A. and Briand, L. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conf. on Software Engineering, Association for Computing Machinery* (2011), 1–10; <https://doi.org/10.1145/1985793.1985795>.
2. Artifact review and badging. Association for Computing Machinery; <https://bit.ly/3ValQz6>.
3. Badampudi, D., Wohlin, C., and Gorschek, T. Contextualizing research evidence through knowledge translation in software engineering. *ACM International Conference Proceeding Series* (2019), 306–311; <https://doi.org/10.1145/3319008.3319358>.
4. Baker, M. and Penny, D. Is there a reproducibility crisis? *Nature* 533, 7604 (2016), 452–454; <https://doi.org/10.1038/533452A>.
5. Baldassarre, M.T., Caivano, D., Romano, S., and Scanniello, G. Software models for source code maintainability: A systematic literature review. *2019 45th Euromicro Conf. on Software Engineering and Advanced Applications*, IEEE, 252–259.
6. Bentley, J.L. Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18, 9 (September 1975), 509–517; <https://doi.org/10.1145/361002.361007>.
7. Bernal-Cárdenas, C. et al. Translating video recordings of mobile app usages into replayable scenarios. In *Proceedings of the ACM/IEEE 42nd Intern. Conf. on Software Engineering* (2020); <https://doi.org/10.1145/3377811.3380328>.

8. Childers, B.R. and Chrysanthis, P.K. Artifact evaluation: Is it a real incentive? *2017 IEEE 13th Intern. Conf. on e-Science*, 488–489; <https://bit.ly/3hFa6Z4>.
9. Collberg, C. and Proebsting, T.A. Repeatability in computer systems research. *Communications of the ACM* 59, 3 (2016), 62–69; <https://bit.ly/3HKABXx>.
10. da Silva, F.Q.B. Replication of empirical studies in software engineering research: A systematic mapping study. *Empirical Software Engineering* (September 2012); <https://doi.org/10.1007/s10664-012-9227-7>.
11. DeVito, N.J., Bacon, S., and Goldacre, B. Compliance with legal requirement to report clinical trial results on [ClinicalTrials.gov](https://clinicaltrials.gov): A cohort study. *The Lancet* 395, 10221 (2020), 361–369; <https://bit.ly/3PDwIFM>.
12. Felizardo, K.R. et al. Secondary studies in the academic context: A systematic mapping and survey. *J. of Systems and Software* 170 (2020), 110734.
13. Fleiss, J.L. Measuring nominal scale agreement among many raters. *Psychological Bulletin* 76, 5 (1971), 378.
14. Hermann, B., Winter, S., and Siegmund, J. Community expectations for research artifacts and evaluation processes. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering*, Association for Computing Machinery (2020), 469–480; <https://doi.org/10.1145/3368089.3409767>.
15. Heumüller, R., Nielebock, S., Krüger, J., and Ortmeier, F. Publish or perish, but do not forget your software artifacts. *Empirical Software Engineering* (2020); <https://doi.org/10.1007/s10664-020-09851-6>.
16. Kalliamvakou, E. et al. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conf. on Mining Software Repositories—MSR 2014*, ACM Press; <https://doi.org/10.1145/2597073.2597074>.
17. Krishnamurthi, S. and Vitek, J. The real software crisis: Repeatability as a core value. *Communications of the ACM* 58, 3 (February 2015), 34–36; <https://doi.org/10.1145/2658987>.
18. Lessmann, S., Baesens, B., Mues, C., and Pietsch, S. Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE Transactions on Software Engineering* 34, 4 (2008), 485–496; <https://bit.ly/3jdmrnF>.
19. Mathew, G., Agrawal, A., and Menzies, T. Finding trends in software research. *IEEE Transactions on Software Engineering* (2018), 1–1; <https://bit.ly/3hwwUSE>.
20. Mayo, D.G. *Statistical Inference as Severe Testing: How to Get Beyond the Statistics Wars*. Cambridge University Press. 2018.
21. Menzies, T. Guest editorial for the Special Section on Best Papers from the 2011 Conference on Predictive Models in Software Engineering (PROMISE). *Information and Software Technology* 55, 8 (2013), 1477–1478.
22. Moran, K. et al. Machine learning-based prototyping of graphical user interfaces for mobile apps. *IEEE Transactions on Software Engineering* 46, 2 (2020), 196–221; <https://doi.org/10.1109/tse.2018.2844788>.
23. Popper, K. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge (1963).
24. Pritchard, A. Statistical bibliography or bibliometrics? *J. of Documentation* 25, 4 (1969), 348–349.
25. Rousseau, R. Forgotten founder of bibliometrics. *Nature* 510 (2014).
26. Russakovsky, O. et al. ImageNet large scale visual recognition challenge. *Intern. J. of Computer Vision* 115, 3 (2015), 211–252; <https://bit.ly/3WkR6hc>.
27. Santos, A., Vegas, S., Oivo, M., and Juristo, N. Comparing the results of replications in software engineering. *Empirical Software Engineering* 26, 2 (February 2021); <https://doi.org/10.1007/s10664-020-09907-7>.

28. Schimmack, U. A meta-psychological perspective on the decade of replication failures in social psychology. *Canadian Psychology* 61, 4 (November 2020), 364–376; <https://doi.org/10.1037/cap0000246>.

29. Zhou, Z.Q., Tse, T.H., and Witheridge, M. Metamorphic robustness testing: Exposing hidden defects in citation statistics and journal impact factors. *IEEE Transactions on Software Engineering* 47, 6 (2021), 1164–1183; <https://bit.ly/3VcqG05>.

Footnotes

a. That team included the authors of this paper plus Jacky Keung from City University, Hong Kong; Greg Gay from Chalmers University, Sweden; Burak Turhan from Oulu University, Finland; and Aldeida Aleti from Monash University, Australia. We gratefully acknowledge their work and the work of their graduate students. We especially call out the work of Afonso Fontes from Chalmers University, Sweden.

b. For example, see <https://bit.ly/3Vbtqef>

c. For example, see the 22 types of potentially reusable items at <https://bit.ly/3FljMtG>

d. See <https://bit.ly/3vlgxbx/>

e. As of December 10, 2020, that search returns 2,535 software engineering papers with artifact badges. Of these, 43% are available, 30% are functional, 20% are reusable, 5% are reproduced, and 2% are replicated artifacts.

f. See <https://bit.ly/3hDc2Bf>

About the Authors

Maria Teresa Baldassarre is a professor at the University of Bari, Italy.

Neil Ernst is a professor at the University of Victoria, Canada.

Ben Hermann is a professor at the Technische Universität Dortmund, Germany.

Tim Menzies (tjmenzie@ncsu.edu) is a professor at North Carolina State University, Raleigh, NC, USA.

Rahul Yedida is a Ph.D. candidate at North Carolina State University, Raleigh, NC, USA.

Submit an Article to CACM

CACM welcomes unsolicited [submissions](#) on topics of relevance and value to the computing community.

©2023 ACM 0001-0782/23/02

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and full citation on the first page. Copyright for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or fee. Request permission to publish from permissions@acm.org or fax (212) 869-0481.

The Digital Library is published by the Association for Computing Machinery. Copyright © 2023 ACM, Inc.

Join the Discussion (0)

